

División de datos y validación cruzada

(Dataset [disponible en Kaggle](#))

Presentación del dataset

En este cuaderno volvemos a trabajar con el dataset "**Popular Video Games 1980-2023**", publicado en Kaggle por el usuario **arnabchaki**, por lo que no vamos a abundar en su descripción.

Importar librerías para trabajar con los datos

Además de las librerías que ya conocemos `Pandas` `NumPy` `Matplotlib` `SciPy` etc) e importamos en la celda siguiente, más adelante importaremos `Scikit-Learn`

En este bloque importamos las bibliotecas que vamos a usar a lo largo del cuaderno. Algunas nos sirven para cargar y transformar los datos, y otras para dividir el dataset, entrenar modelos y medir su rendimiento.

Cómo interpretar el resultado

Si la celda se ejecuta sin errores, significa que el entorno quedó listo para trabajar. En esta etapa no buscamos resultados numéricos, sino preparar las herramientas necesarias para las siguientes actividades.

```
# Librerías para el manejo de datos
import pandas as pd
import numpy as np

# Librerías para creación de gráficos
import matplotlib.pyplot as plt

# Librerías para pruebas estadísticas
from scipy import stats as st
import math as mt

# Librería para importar directamente desde Kaggle
import kagglehub

# Extra: acceso a comandos del Sistema Operativo
import os
```

Cargar el dataset

Aquí cargamos el dataset de videojuegos. Primero se intenta abrir el archivo `games.csv` si ya está disponible en el entorno. Si no aparece, se descarga automáticamente desde Kaggle y luego se carga en un `DataFrame` llamado `games`.

El mensaje final nos indica de dónde se obtuvo el archivo. Lo importante es que, al terminar, el `DataFrame` `games` quede disponible para comenzar la exploración y el modelado.

```
# Intentamos abrir el archivo si ya está disponible en el entorno
try:
    games = pd.read_csv("games.csv")
    print("Dataset cargado desde el archivo local 'games.csv'")
except Exception:
    print("No se encontró 'games.csv' en el entorno local.")
    print("Descargando dataset desde Kaggle...")

    ruta_dataset = kagglehub.dataset_download("arnabchaki/popular-video-ga
    ruta_csv = os.path.join(ruta_dataset, "games.csv")

    games = pd.read_csv(ruta_csv)
    print("Dataset descargado y cargado correctamente desde Kaggle")
```

No se encontró 'games.csv' en el entorno local.
Descargando dataset desde Kaggle...
Using Colab cache for faster access to the 'popular-video-games-1980-2023' dataset.
Dataset descargado y cargado correctamente desde Kaggle

Exploración inicial de los datos

En esta primera exploración nos concentramos solo en las columnas que podrían servirnos para el modelo. Observamos `Rating` y varias columnas con cantidades de interacciones de los usuarios, como la cantidad de reseñas, partidas o juegos en lista de espera.

Cómo interpretar el resultado

La tabla nos permite ver **cómo vienen escritos los datos**. En este caso aparecen valores como `3.9K` o `17K`, que todavía están en formato de texto. Además, el conteo de valores nulos nos muestra si alguna columna necesita limpieza antes de usarse. Esta inspección previa es importante porque un modelo no puede trabajar correctamente con valores ambiguos o mal tipados.

```
# =====  
# INSPECCIONAMOS LAS COLUMNAS QUE PODRÍAN SERVIRNOS  
# =====  
  
columnas_interes = [  
    "Rating",  
    "Times Listed",  
    "Number of Reviews",  
    "Plays",  
    "Playing",  
    "Backlogs",  
    "Wishlist"  
]  
  
display(games[columnas_interes].head(10))  
print(games[columnas_interes].isnull().sum())
```

	Rating	Times Listed	Number of Reviews	Plays	Playing	Backlogs	Wishlist
0	4.5	3.9K	3.9K	17K	3.8K	4.6K	4.8K
1	4.3	2.9K	2.9K	21K	3.2K	6.3K	3.6K
2	4.4	4.3K	4.3K	30K	2.5K	5K	2.6K
3	4.2	3.5K	3.5K	28K	679	4.9K	1.8K
4	4.4	3K	3K	21K	2.4K	8.3K	2.3K
5	4.3	2.3K	2.3K	33K	1.8K	1.1K	230
6	4.2	1.6K	1.6K	7.2K	1.1K	4.5K	3.8K
7	4.3	2.1K	2.1K	9.2K	759	3.4K	3.3K
8	3.0	867	867	25K	470	776	126
9	4.3	2.9K	2.9K	18K	1.1K	6.2K	3.6K

```

Rating          13
Times Listed    0
Number of Reviews 0
Plays           0
Playing         0
Backlogs        0
Wishlist        0
dtype: int64

```

Convertir texto con formato a número

En este bloque convertimos a números las columnas que vienen como texto con sufijos como **K** . Por ejemplo, **3.9K** pasa a **3900.0** . Para eso definimos una función sencilla y la aplicamos sobre varias columnas del dataset.

Después de la conversión, esperamos ver números decimales en lugar de textos y tipos de dato **float64** . Eso nos confirma que esas variables ya quedaron listas para ser usadas como **features** del modelo.

```

# =====
# CONVERTIMOS COLUMNAS CON FORMATO TIPO "K" A VALORES NUMÉRICOS
# =====

def convertir_k_a_numero(valor):
    """
    Convierte textos como '3.9K', '17K' o '230' a números.
    Si el valor ya es numérico, lo devuelve.
    """
    if pd.isna(valor):
        return None

    valor = str(valor).strip().upper()

```

```

    if valor.endswith("K"):
        return float(valor[:-1]) * 1000
    else:
        return float(valor)

columnas_numericas_texto = [
    "Times Listed",
    "Number of Reviews",
    "Plays",
    "Playing",
    "Backlogs",
    "Wishlist"
]

# Hacemos una copia para trabajar con seguridad
games_transformado = games.copy()

# Aplicamos la conversión
for columna in columnas_numericas_texto:
    games_transformado[columna] = games_transformado[columna].apply(conver

# Verificamos el resultado
display(games_transformado[columnas_numericas_texto].head(10))
print(games_transformado[columnas_numericas_texto].dtypes)

```

	Times Listed	Number of Reviews	Plays	Playing	Backlogs	Wishlist
0	3900.0	3900.0	17000.0	3800.0	4600.0	4800.0
1	2900.0	2900.0	21000.0	3200.0	6300.0	3600.0
2	4300.0	4300.0	30000.0	2500.0	5000.0	2600.0
3	3500.0	3500.0	28000.0	679.0	4900.0	1800.0
4	3000.0	3000.0	21000.0	2400.0	8300.0	2300.0
5	2300.0	2300.0	33000.0	1800.0	1100.0	230.0
6	1600.0	1600.0	7200.0	1100.0	4500.0	3800.0
7	2100.0	2100.0	9200.0	759.0	3400.0	3300.0
8	867.0	867.0	25000.0	470.0	776.0	126.0
9	2900.0	2900.0	18000.0	1100.0	6200.0	3600.0

```

Times Listed      float64
Number of Reviews float64
Plays             float64
Playing           float64
Backlogs          float64
Wishlist          float64
dtype: object

```

Construimos la variable a predecir

Aquí construimos la **variable objetivo** que el modelo intentará predecir. Tomamos la columna `Rating` y la transformamos en una clasificación binaria: juegos con alta valoración y juegos con valoración menor. Antes de hacerlo, eliminamos las filas donde `Rating` es nulo.

El conteo final nos muestra cuántos juegos quedaron en cada clase. No hace falta que ambas clases tengan exactamente el mismo tamaño, pero sí conviene verificar que no haya un desbalance extremo. Si una de las dos clases fuera muy pequeña, la clasificación sería más difícil de interpretar.

```
# =====  
# CREAMOS UNA VARIABLE OBJETIVO BINARIA A PARTIR DE RATING  
# =====  
  
# Eliminamos filas con Rating nulo  
games_modelo = games_transformado.dropna(subset=["Rating"]).copy()  
  
# Creamos la variable objetivo:  
# 1 = juego con alta valoración  
# 0 = juego con valoración menor  
games_modelo["Alta_Valoracion"] = (games_modelo["Rating"] >= 4.0).astype(int)  
  
# Vemos cuántos casos hay de cada clase  
print("Distribución de la variable objetivo:")  
print(games_modelo["Alta_Valoracion"].value_counts())  
  
print("\nProporciones:")  
print(games_modelo["Alta_Valoracion"].value_counts(normalize=True))
```

Distribución de la variable objetivo:

Alta_Valoracion

0 918

1 581

Name: count, dtype: int64

Proporciones:

Alta_Valoracion

0 0.612408

1 0.387592

Name: proportion, dtype: float64

Los resultados muestran que, después de eliminar los valores nulos de `Rating`, quedaron 1499 juegos con una distribución bastante razonable entre las dos clases. Aproximadamente el 61,2% de los casos quedó en la categoría 0 y el 38,8% en la categoría 1.

Esto indica que la variable objetivo no está perfectamente balanceada, pero tampoco presenta un desbalance extremo. Por lo tanto, se puede trabajar con

esta clasificación sin grandes problemas: el modelo no va a estar “forzado” a predecir casi siempre la misma clase.

También es importante decir que esta nueva variable simplifica el problema: en lugar de predecir un valor exacto de ‘Rating’, el modelo solo tendrá que decidir si un juego entra o no en la categoría de ‘alta valoración’.

Division Train/Test

En esta parte separamos las variables predictoras (*X*) de la variable objetivo (*y*) y luego dividimos los datos en dos conjuntos: uno de entrenamiento y otro de prueba. Usamos una división 80/20 y además aplicamos `stratify=y` para conservar la proporción de clases en ambos conjuntos.

Cómo interpretar el resultado

Las formas de `X_train` , `X_test` , `y_train` y `y_test` muestran cuántas filas quedaron en cada subconjunto. Luego, las proporciones de clases permiten comprobar que entrenamiento y prueba conservan una distribución muy parecida. Eso es importante porque evita que una partición quede sesgada respecto de la otra.

```
# =====  
# SEPARAMOS VARIABLES PREDICTORAS (X) Y VARIABLE OBJETIVO (y)  
# =====  
  
from sklearn.model_selection import train_test_split  
  
features = [  
    "Times Listed",  
    "Number of Reviews",  
    "Plays",  
    "Playing",  
    "Backlogs",  
    "Wishlist"  
]  
  
X = games_modelo[features]  
y = games_modelo["Alta_Valoracion"]  
  
# Dividimos en entrenamiento y prueba  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

```
print("Forma de X_train:", X_train.shape)
print("Forma de X_test :", X_test.shape)
print("Forma de y_train:", y_train.shape)
print("Forma de y_test :", y_test.shape)

print("\nProporción de clases en y_train:")
print(y_train.value_counts(normalize=True))

print("\nProporción de clases en y_test:")
print(y_test.value_counts(normalize=True))
```

```
Forma de X_train: (1199, 6)
Forma de X_test : (300, 6)
Forma de y_train: (1199,)
Forma de y_test : (300,)
```

```
Proporción de clases en y_train:
Alta_Valoracion
0      0.612177
1      0.387823
Name: proportion, dtype: float64
```

```
Proporción de clases en y_test:
Alta_Valoracion
0      0.613333
1      0.386667
Name: proportion, dtype: float64
```

Los resultados confirman que la división del dataset se realizó correctamente. El conjunto de entrenamiento quedó con 1199 casos y el de prueba con 300, lo que se corresponde con una partición aproximada del 80% para entrenar y 20% para evaluar.

También vemos que `X_train` y `X_test` tienen 6 columnas, porque contienen las variables predictoras seleccionadas. En cambio, `y_train` e `y_test` muestran una forma con un solo valor, ya que `y` es una única serie con la variable objetivo.

Lo más importante es que las proporciones de clases en entrenamiento y prueba son casi idénticas. En ambos conjuntos, la clase `0` representa alrededor del 61% y la clase `1` cerca del 39%. Esto indica que el uso de `stratify=y` funcionó como esperábamos: ambas particiones conservan una distribución muy similar a la del dataset original.

Esto es importante porque permite evaluar el modelo en condiciones más justas. Si una de las particiones tuviera una distribución muy distinta, los resultados podrían verse afectados no por la calidad del modelo, sino por una división poco representativa de los datos.

Entrenamos un modelo (DecisionTree)

Aquí entrenamos un primer modelo de clasificación, en este caso un árbol de decisión sencillo. El objetivo no es profundizar en su funcionamiento interno, sino usarlo como herramienta para mostrar cómo se entrena un modelo y cómo se evalúa sobre datos conocidos y no vistos.

Cómo interpretar el resultado

Se calcula la **accuracy** tanto en entrenamiento como en prueba. Lo esperable es que el valor de entrenamiento sea algo mayor, porque el modelo aprendió justamente con esos datos. La comparación entre ambos resultados empieza a darnos pistas sobre la capacidad de generalización del modelo.

```
# =====
# ENTRENAMOS UN PRIMER MODELO DE CLASIFICACIÓN
# =====

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Creamos el modelo
modelo_arbol = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

# Entrenamos el modelo
modelo_arbol.fit(X_train, y_train)

# Predicciones sobre entrenamiento y prueba
y_pred_train = modelo_arbol.predict(X_train)
y_pred_test = modelo_arbol.predict(X_test)

# Calculamos accuracy
acc_train = accuracy_score(y_train, y_pred_train)
acc_test = accuracy_score(y_test, y_pred_test)

print("Accuracy en entrenamiento:", acc_train)
print("Accuracy en prueba:", acc_test)
```

Accuracy en entrenamiento: 0.7447873227689742
Accuracy en prueba: 0.6833333333333333

El modelo obtuvo una **accuracy de aproximadamente 0,745 en entrenamiento** y **0,683 en prueba**. Esto significa que clasificó correctamente cerca del 74,5% de los casos con los que aprendió y alrededor del 68,3% de los casos nuevos.

La diferencia entre ambos valores es esperable: el rendimiento suele ser mejor en entrenamiento porque el modelo ya vio esos ejemplos durante el aprendizaje. En cambio, el conjunto de prueba representa una situación más exigente, ya que contiene datos no vistos.

En este caso, la diferencia entre entrenamiento y prueba existe, pero no es exageradamente grande. Esto sugiere que el modelo logra cierta capacidad de generalización, aunque todavía no podemos sacar conclusiones definitivas con una sola configuración. Más adelante, al comparar árboles con distintas profundidades, podremos ver mejor cuándo un modelo resulta demasiado simple, cuándo está más equilibrado y cuándo empieza a sobreajustarse.

Reentrenamos con distintos hiperparámetros

Ahora repetimos el entrenamiento varias veces, pero cambiando la profundidad máxima del árbol. De esta manera comparamos modelos con distinta complejidad: algunos muy simples, otros intermedios y otros mucho más complejos.

Cómo interpretar el resultado

La tabla resultante permite observar cómo cambian los desempeños en entrenamiento y en prueba. Si un modelo rinde bajo en ambos casos, puede estar quedándose corto y caer en **underfitting**. Si en cambio rinde muchísimo mejor en entrenamiento que en prueba, aparece una posible señal de **overfitting**.

```
# =====  
# COMPARAMOS ÁRBOLES CON DISTINTA COMPLEJIDAD  
# =====  
  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score  
import pandas as pd  
  
profundidades = [1, 3, 10, None]  
resultados = []  
  
for profundidad in profundidades:  
    modelo = DecisionTreeClassifier(  
        max_depth=profundidad,  
        random_state=42  
    )  
  
    modelo.fit(X_train, y_train)  
  
    y_pred_train = modelo.predict(X_train)
```

```

y_pred_test = modelo.predict(X_test)

acc_train = accuracy_score(y_train, y_pred_train)
acc_test = accuracy_score(y_test, y_pred_test)

resultados.append({
    "max_depth": profundidad,
    "accuracy_train": acc_train,
    "accuracy_test": acc_test
})

df_resultados = pd.DataFrame(resultados)
display(df_resultados)

```

	max_depth	accuracy_train	accuracy_test
0	1.0	0.717264	0.646667
1	3.0	0.744787	0.683333
2	10.0	0.970809	0.770000
3	NaN	1.000000	0.780000

La tabla muestra con claridad que, a medida que aumenta la profundidad del árbol, el modelo se vuelve más complejo y mejora su rendimiento sobre los datos de entrenamiento. Esto era esperable, porque un árbol más profundo tiene más capacidad para ajustarse a los patrones del dataset.

Con `max_depth = 1`, el modelo es muy simple y obtiene valores relativamente bajos tanto en entrenamiento como en prueba. Esto sugiere que podría estar ocurriendo **underfitting**, es decir, que el modelo no tiene suficiente complejidad como para captar bien la estructura de los datos.

Con `max_depth = 3`, el rendimiento mejora un poco en ambos conjuntos. Sigue siendo un modelo relativamente simple, pero logra captar mejor algunos patrones sin mostrar todavía una diferencia muy grande entre entrenamiento y prueba.

Con `max_depth = 10`, el accuracy en entrenamiento sube mucho, hasta cerca de 0,97, y también mejora claramente en prueba. Esto indica que el modelo está aprendiendo bastante más de los datos disponibles.

Finalmente, con `max_depth = None`, el árbol crece sin límite y alcanza una accuracy perfecta en entrenamiento. Eso significa que logra ajustarse completamente a los ejemplos con los que aprendió. Sin embargo, en prueba el resultado no sube en la misma proporción. Esta diferencia es una primera señal de posible **overfitting**: el modelo se adapta demasiado a los datos de entrenamiento y su capacidad de generalización puede volverse más frágil.

En conjunto, esta comparación muestra por qué la complejidad del modelo es importante. Un modelo demasiado simple puede quedarse corto, pero uno excesivamente complejo puede comenzar a memorizar en lugar de generalizar.

Entrenamiento vs. Prueba

En este paso agregamos una columna llamada `brecha`, que calcula la diferencia entre la accuracy de entrenamiento y la accuracy de prueba.

Esta columna ayuda a visualizar mejor la relación entre aprendizaje y generalización. Una brecha pequeña suele indicar un comportamiento más equilibrado. En cambio, una brecha grande puede sugerir que el modelo se ajustó demasiado a los datos de entrenamiento y ya no responde igual de bien ante datos nuevos.

```
# =====  
# CALCULAMOS LA BRECHA ENTRE ENTRENAMIENTO Y PRUEBA  
# =====  
  
df_resultados["brecha"] = df_resultados["accuracy_train"] - df_resultados["accuracy_test"]  
display(df_resultados)
```

	max_depth	accuracy_train	accuracy_test	brecha
0	1.0	0.717264	0.646667	0.070598
1	3.0	0.744787	0.683333	0.061454
2	10.0	0.970809	0.770000	0.200809
3	NaN	1.000000	0.780000	0.220000

La nueva columna `brecha` permite ver con más claridad cuánto se separa el rendimiento del modelo entre entrenamiento y prueba. Resume, en un solo valor, qué tan parecido o qué tan distinto es el comportamiento del modelo en datos conocidos y en datos no vistos.

En los árboles más simples, como `max_depth = 1` y `max_depth = 3`, la brecha es relativamente pequeña. Esto sugiere que el modelo mantiene un comportamiento parecido en ambos conjuntos. Sin embargo, como ya vimos antes, esos modelos también tienen un rendimiento general más bajo, por lo que una brecha pequeña no siempre significa que el modelo sea bueno: también puede indicar que está aprendiendo poco.

En cambio, cuando la profundidad aumenta a `10` o queda sin límite (`None`), la brecha crece mucho. En estos casos, el modelo rinde extremadamente bien en entrenamiento, pero la mejora en prueba es bastante menor. Esto es una señal típica de **overfitting**: el modelo se ajusta demasiado a los datos con los que aprendió y pierde parte de su capacidad para generalizar.

Por eso, esta columna resulta muy útil para la interpretación. No alcanza con mirar solo qué modelo tiene mejor accuracy en entrenamiento. También es importante observar si ese buen rendimiento se mantiene cuando el modelo enfrenta datos nuevos.

Validación cruzada K-Fold

Aquí aplicamos **validación cruzada K-Fold** con 5 pliegues. En lugar de depender de una sola partición train/test, el modelo se entrena y evalúa varias veces, usando en cada vuelta un subconjunto distinto como prueba.

Los cinco valores de accuracy muestran el rendimiento obtenido en cada pliegue. El promedio resume el desempeño general del modelo, mientras que el desvío estándar indica cuánto varían esos resultados entre una partición y otra. Cuanto menor sea esa variación, más estable tiende a ser la evaluación.

```
# =====  
# APLICAMOS VALIDACIÓN CRUZADA (K-FOLD)  
# =====  
  
from sklearn.model_selection import cross_val_score  
from sklearn.tree import DecisionTreeClassifier  
  
modelo_cv = DecisionTreeClassifier(max_depth=3, random_state=42)  
  
scores_cv = cross_val_score(  
    modelo_cv,  
    X,  
    y,  
    cv=5,  
    scoring="accuracy"  
)  
  
print("Accuracy en cada fold:")  
print(scores_cv)  
  
print("\nAccuracy promedio:")  
print(scores_cv.mean())  
  
print("\nDesvío estándar:")  
print(scores_cv.std())
```

Accuracy en cada fold:
[0.62333333 0.62 0.53 0.75333333 0.61204013]

Accuracy promedio:
0.6277413600891861

Desvío estándar:
0.0716276553839209

Los resultados muestran que el modelo no obtiene exactamente el mismo desempeño en todos los pliegues. Esto es normal, porque en cada fold cambia qué subconjunto del dataset se usa como prueba. En este caso, las accuracies van aproximadamente desde 0.53 hasta 0.75, lo que indica que algunas particiones resultan más favorables que otras para el modelo.

El **accuracy promedio**, cercano a 0.628, ofrece una estimación más robusta del rendimiento general que una única partición train/test. En lugar de depender de una sola división del dataset, este valor resume el comportamiento del modelo a lo largo de cinco evaluaciones distintas.

El **desvío estándar**, de aproximadamente 0.072, muestra que existe una variación moderada entre los folds. Esto sugiere que el rendimiento del modelo no es completamente estable y que depende en cierta medida de cómo se repartan los datos. Justamente por eso la validación cruzada resulta valiosa: permite obtener una visión más completa y menos dependiente del azar de una sola partición.

Estos resultados refuerzan la idea de que evaluar un modelo con varias divisiones del dataset brinda una medida más confiable de su capacidad de generalización.

Validación cruzada K-Fold con distintos hiperparámetros

En esta celda repetimos la validación cruzada para árboles con distintas profundidades. Así podemos comparar la complejidad de los modelos, pero ahora con una evaluación más robusta que la simple partición train/test.

Cómo interpretar el resultado

La columna `accuracy_promedio_cv` permite ver qué profundidad funciona mejor en promedio. La columna `desvio_std_cv` muestra si ese rendimiento es más o menos estable a lo largo de los distintos pliegues. Esto ayuda a comparar modelos de forma más confiable.

```
# =====
# COMPARAMOS DISTINTAS PROFUNDIDADES CON VALIDACIÓN CRUZADA
# =====

from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

profundidades = [1, 3, 10, None]
resultados_cv = []

for profundidad in profundidades:
    modelo = DecisionTreeClassifier(max_depth=profundidad, random_state=42)

    scores = cross_val_score(
        modelo,
        X,
        y,
        cv=5,
        scoring="accuracy"
    )

    resultados_cv.append({
        "max_depth": profundidad,
        "accuracy_promedio_cv": scores.mean(),
        "desvio_std_cv": scores.std()
    })

df_resultados_cv = pd.DataFrame(resultados_cv)
display(df_resultados_cv)
```

	max_depth	accuracy_promedio_cv	desvio_std_cv
0	1.0	0.610408	0.076104
1	3.0	0.627741	0.071628
2	10.0	0.749124	0.094124
3	NaN	0.803148	0.081095

La tabla muestra que, en esta práctica, el rendimiento promedio del modelo mejora a medida que aumenta la profundidad del árbol. Los árboles más simples, como los de profundidad 1 y 3, obtienen accuracies promedio más bajas, lo que sugiere que podrían estar quedándose cortos para captar bien los patrones del problema.

En cambio, los árboles de mayor profundidad, especialmente `max_depth = 10` y `max_depth = None`, alcanzan accuracies promedio bastante más altos. Esto indica que los modelos más complejos logran aprender mejor a partir de las variables disponibles.

También conviene observar la columna `desvio_std_cv` . En todos los casos hay cierta variación entre los folds, aunque no es exageradamente grande. Eso significa que el rendimiento cambia algo según la partición utilizada, pero mantiene una tendencia general bastante consistente.

Un punto interesante es que, en esta validación cruzada, el árbol sin límite de profundidad (`None`) obtiene el mejor promedio. Esto muestra que un modelo más complejo no siempre empeora automáticamente al evaluarlo varias veces. Sin embargo, al compararlo con los resultados anteriores de entrenamiento y prueba, sigue siendo importante recordar que también era el que mostraba la mayor brecha entre ambos. Por eso, la interpretación debe hacerse considerando varios indicadores a la vez.

Esta tabla refuerza dos ideas importantes: la complejidad del modelo influye mucho en su desempeño, y la validación cruzada permite comparar esas configuraciones de una manera más confiable.

Resumen de resultados

Por último, reunimos en una misma tabla los resultados de la partición train/test y los de la validación cruzada. Esto nos permite observar toda la información relevante en un solo lugar.

Esta tabla final sirve como síntesis de la práctica. En ella se puede comparar el rendimiento en entrenamiento, en prueba, la brecha entre ambos y el promedio obtenido con validación cruzada. A partir de esta comparación podemos discutir con más fundamento si un modelo parece demasiado simple, demasiado complejo o razonablemente equilibrado.

```
# =====  
# UNIMOS LOS RESULTADOS DE TRAIN/TEST Y VALIDACIÓN CRUZADA  
# =====  
  
df_comparacion_final = pd.merge(  
    df_resultados,  
    df_resultados_cv,  
    on="max_depth"  
)  
  
display(df_comparacion_final)
```


	max_depth	accuracy_train	accuracy_test	brecha	accuracy_promedio_cv	desvio_std
0	1.0	0.717264	0.646667	0.070598	0.610408	0.076
1	3.0	0.744787	0.683333	0.061454	0.627741	0.071
2	10.0	0.970809	0.770000	0.200809	0.749124	0.094
3	NaN	1.000000	0.780000	0.220000	0.803148	0.081

Esta tabla final permite reunir en una sola vista las dos formas de evaluación que trabajamos en la práctica: la partición simple train/test y la validación cruzada. Gracias a eso, podemos comparar no solo qué modelo rinde mejor, sino también cómo cambia ese rendimiento según el criterio que usemos para evaluarlo.

Los árboles más simples, como los de profundidad 1 y 3, muestran accuracies relativamente bajas tanto en prueba como en validación cruzada. Además, sus brechas entre entrenamiento y prueba son pequeñas. Esto sugiere que no están sobreajustando, pero también que su capacidad de aprendizaje es limitada. En otras palabras, son modelos que podrían estar cerca del **underfitting**.

A medida que aumenta la profundidad, el rendimiento mejora claramente. Con `max_depth = 10`, el modelo logra accuracies mucho más altas, tanto en prueba como en validación cruzada. Sin embargo, también aparece una brecha bastante mayor entre entrenamiento y prueba, lo que indica que el modelo empieza a ajustarse mucho más a los datos de entrenamiento.

El caso más extremo es `max_depth = None`. Allí el árbol alcanza una accuracy perfecta en entrenamiento (1.00), lo que significa que logra ajustarse por completo a los datos con los que aprendió. Al mismo tiempo, es el modelo con mayor brecha, lo que constituye una señal clara de **overfitting**. Sin embargo, también obtiene el mejor promedio en validación cruzada, lo cual muestra que la interpretación no siempre es completamente simple: un modelo muy complejo puede exhibir señales de sobreajuste y aun así mantener un buen rendimiento promedio.

En conjunto, esta síntesis refuerza una idea importante de la clase: para evaluar un modelo no alcanza con mirar una sola métrica ni una sola partición del dataset. Conviene observar varias evidencias a la vez, como el rendimiento en entrenamiento, en prueba, la brecha entre ambos y el promedio obtenido mediante validación cruzada. Solo así podemos formar una idea más completa sobre si el modelo está aprendiendo poco, ajustándose demasiado o alcanzando un equilibrio razonable.

Conclusiones de la práctica

En esta actividad trabajamos con un dataset real de videojuegos para mostrar algunas ideas centrales de la evaluación de modelos en machine learning.

Primero, dividimos los datos en conjunto de entrenamiento y conjunto de prueba. Esto nos permitió entrenar el modelo con una parte del dataset y evaluarlo con datos no vistos, logrando una medición más honesta de su desempeño.

Luego entrenamos árboles de decisión con distintos niveles de complejidad. Observamos que los modelos muy simples tienden a rendir peor tanto en entrenamiento como en prueba, lo que puede asociarse con **underfitting**. En cambio, los modelos más complejos logran un mejor desempeño, aunque también muestran una mayor diferencia entre entrenamiento y prueba, lo que puede interpretarse como una señal de **overfitting**.

Finalmente, aplicamos **validación cruzada (K-Fold Cross Validation)** para obtener una evaluación más robusta. En lugar de depender de una sola partición del dataset, el modelo fue evaluado en varios pliegues, lo que nos dio una estimación más estable de su rendimiento general.

En síntesis, esta práctica mostró que:

- dividir los datos es fundamental para evaluar un modelo correctamente;
- la complejidad del modelo influye en su capacidad de aprender y generalizar;
- y la validación cruzada ayuda a obtener conclusiones más confiables.

Notas

En esta actividad aparecieron algunos detalles interesantes que vale la pena remarcar, porque muestran que en machine learning los resultados no siempre son perfectamente “limpios” o ideales.

1. Un modelo puede no ser tan útil como parece

Aunque algunos árboles obtuvieron accuracies relativamente aceptables, eso no significa automáticamente que sean modelos realmente buenos o útiles. Por ejemplo, en problemas donde una clase es más frecuente que la otra, un

modelo muy simple podría acertar bastante solo por seguir la mayoría. Por eso, en la práctica real, siempre conviene preguntarse si el modelo realmente está aprendiendo algo valioso o si solo está aprovechando una regularidad muy básica del dataset.

2. El overfitting no siempre aparece de forma perfecta

En teoría solemos decir que un modelo con overfitting rinde excelente en entrenamiento y peor en prueba. En esta práctica eso se vio en parte: los árboles más profundos lograron resultados altísimos en entrenamiento y mostraron brechas mayores respecto de prueba.

Sin embargo, también ocurrió algo interesante: el árbol más complejo siguió obteniendo un buen promedio en validación cruzada. Esto muestra que la interpretación no siempre es absoluta. Un modelo puede exhibir señales de sobreajuste y, aun así, seguir siendo competitivo en algunas evaluaciones. Por eso conviene mirar varios indicadores al mismo tiempo, y no quedarse con uno solo.

3. El valor NaN en max_depth no significa “dato faltante” del dataset

En las tablas, el árbol cuya profundidad se definió como `None` aparece mostrado como `NaN`. Esto no quiere decir que falte un dato en el dataset ni que haya un error en el modelo. Ocurre porque `pandas`, al construir la tabla, convierte esa columna a un tipo numérico y representa el valor `None` como `NaN`.

Es decir, en este contexto, `NaN` debe interpretarse como: **árbol sin límite de profundidad**.

4. Una sola métrica no cuenta toda la historia

En esta práctica usamos `accuracy` porque es una medida simple y útil para empezar. Pero en problemas reales no siempre alcanza con mirar una sola métrica. Dependiendo del caso, también podría ser importante analizar precisión, recall, F1-score o la matriz de confusión.

5. En ciencia de datos, interpretar también es parte del trabajo

Una idea importante para llevarse de esta actividad es que entrenar un modelo no consiste solo en ejecutar código. También implica interpretar resultados, detectar comportamientos inesperados, reconocer limitaciones y decidir si el modelo realmente sirve para el problema que queremos resolver.

En otras palabras: *en machine learning no siempre “sale todo de una”, y justamente por eso el análisis crítico de los resultados es una parte central del trabajo.*